

How AutoNixNan Works: A Code-Level Account

ganesh@cs.utah.edu
(written with Claude Code)

September 21, 2026

Abstract

This document explains, from the source code, how `bin/autonixnan` (“AutoNixNan”) and the `nixnan.so` NVBit tool it drives actually work. It is not a user guide; it is a line-level account of the mechanism, written after reading every file involved (`bin/autonixnan`, `src/nixnan.cu`, and the eleven supporting headers/translation units under `src/`), and after empirically exercising the tool against several real CUDA programs. Where the code’s behavior diverges from what its own comments or the project README claim, that divergence is called out explicitly and is backed by a repeatable observation, not speculation.

Contents

1	The Big Picture	1
2	Part I — <code>nixnan.so</code>	2
2.1	Initialization: <code>nvbit_at_init</code>	2
2.2	Deciding what to instrument	3
2.3	The opcode-to-operand table	4
2.4	Exception detection, end to end	5
2.5	Memory-instruction checking	7
2.6	The exponent-histogram / bin-threshold mechanism	7
2.7	<code>LOG_KERNELS</code> mode	8
2.8	The (currently) dead <code>NIXNAN_TIMEOUT</code>	8
3	Part II — <code>bin/autonixnan</code>	9
3.1	Phase 1: <code>collect_kernels()</code> — the baseline	9
3.2	Report assembly (between phases 1 and 2)	9
3.3	Phase 2: <code>find_extreme_values()</code> — the exponent scan	9
3.4	Phase 3: <code>run_instrumented()</code> — the deep dive	10
3.5	Timeout semantics, summarized	11
4	Empirically Observed Limitations	11
5	Practical Summary	11

1 The Big Picture

AutoNixNan is not one program; it is two, layered:

1. `nixnan.so` (source: `src/*.cu`, `src/*.cuh`, `src/*.h`) — an NVBit tool. NVBit (NVIDIA Binary Instrumentation Tool) is preloaded into a CUDA process via `LD_PRELOAD` and gets a callback on *every* kernel launch, before that kernel’s SASS (compiled machine code) ever executes on the GPU. But the SASS rewrite itself is not per-launch: `instrument_function` (`src/nixnan.cu`) guards it with an `std::unordered_set<CUfunction>` of functions already seen, so it inserts calls to its own device-side functions after (and sometimes before) every floating-point instruction only the *first* time a given kernel function is launched. Every later launch of that same kernel hits the callback again, but `instrument_function` short-circuits immediately and NVBit just runs the already-instrumented SASS via `nvbit_enable_instrumented` — no re-rewriting occurs. This is how `nixnan.so` inspects the actual bits flowing through the GPU’s registers at run time without paying the instrumentation cost on every single launch.
2. `bin/autonixnan` (a 420-line Python 3 script) — a wrapper that never touches SASS or the GPU directly. It only launches the target program multiple times, each time with a different combination of environment variables that steer `nixnan.so`, and stitches the resulting log files into one report.

Everything `autonixnan` does, it does by shelling out to the target program with `LD_PRELOAD=nixnan.so` and a specific environment. There is no IPC between the Python process and the GPU process beyond environment variables in and a text log file out. This matters: every quirk of `autonixnan`’s behavior traces back to either (a) what environment variable combination it chose to set for a given phase, or (b) what `nixnan.so` actually does — which is sometimes *not* what its own `--help` banner or `autonixnan`’s comments say it does (Section 2.8).

2 Part I — `nixnan.so`

2.1 Initialization: `nvbit_at_init`

NVBit calls `nvbit_at_init()` once, before the target’s `main()` runs, but after CUDA driver initialization has begun enough that reading environment variables is possible. `nixnan.cu` uses this hook to read every environment variable it supports via the NVBit-provided `GET_VAR_INT/GET_VAR_STR` macros (defined in `nvbit_release/core/nvbit.h`). Each call to one of these macros *also prints a banner line* to the log stream — this is the “NVBit core environment variables” banner every instrumented run of this session emitted. That banner is not decorative: it is a direct enumeration of every env var the running binary actually reads, in the order it reads them. **If a variable does not appear in the banner, the running binary does not consult it, full stop** — this becomes the key diagnostic in Section 2.8.

Table 1 lists every variable read by `nixnan.cu` itself, in banner order, plus two read by a header it includes (`fp-histogram.cu`) during `nvbit_at_init`’s call to `nixnan::fp_histogram::init()`.

Variable	Default	Effect
<code>INSTR_BEGIN</code>	/ 0	Instruction-index interval to instrument (unused by <code>autonixnan</code>).
<code>INSTR_END</code>	<code>UINT32_MAX</code>	
<code>TOOL_VERBOSE</code>	0	Extra diagnostic printing during instrumentation setup.
<code>ENABLE_FUN_DETAIL</code>	0	Print the full mangled kernel name every time a kernel re-runs, not just the first time.

Variable	Default	Effect
PRINT_ILL_INSTR	0	(declared, not read further in the code paths exercised here).
SAMPLING	0	Instrument only every N th launch of a repeated kernel.
INSTRUMENT_EXCEPTIONS		Master switch for the NaN/Inf/Subnormal/Div-by-0 instrumentation described in Section 2.4.
INSTR_MEM	0	Also instrument 32/64/128-bit global-memory stores (STG) for NaN, Section 2.5.
HISTOGRAM	0	Set implicitly to 1 if BIN_SPEC_FILE is non-empty; enables the exponent-histogram/bin-threshold mechanism, Section 2.6.
BIN_SPEC_FILE	(empty)	Path to a JSON file describing which exponent ranges, per format, should be watched; auto-created as a template and the process exits if the file does not yet exist.
LOGFILE	(empty)	Redirect all <code>nnout()</code> output (everything prefixed <code>#nixnan:</code>) from <code>stderr</code> to this file.
LINE_INFO	1	Attempt to resolve source file:line for each instrumented instruction (wrapped in a SIGSEGV handler — see <code>recording.cu</code>).
LOG_KERNELS	(empty)	Path to a log file; if set, disables all instrumentation and instead just logs every kernel launch and its wall-clock execution time. Section 2.7.
MAX_ERRORS	0 (unlimited)	Self-terminate (SIGINT via <code>nnterminate</code>) after this many exception reports.
TIME_KERNELS	0	Print a “Kernel [...] execution time: N microseconds” line for every kernel launch, independent of LOG_KERNELS.
FUNCTION_WHITELIST	(empty)	Path to a newline-separated file of function names; only these are instrumented.
FUNCTION_BLACKLIST	(empty)	Complement of the above; mutually exclusive with it (the process exits with an error if both are set).

Table 1: Environment variables consulted by the running `nixnan.so` binary, in the order `nvbit_at_init` reads them. Absence from this table (e.g. `NIXNAN_TIMEOUT`) means the variable is *not currently wired up* – see Section 2.8.

2.2 Deciding what to instrument

For every kernel launch, NVBit calls `nvbit_at_cuda_event(ctx, is_exit, cbid, ...)`, once on entry and once on exit, for the CUDA driver calls that launch work (`cuLaunchKernel` and its cooperative variants). On entry (`is_exit == false`):

```
bool enable_instr = instrument_function(ctx, p->f);
if (enable_instr) {
```

```

    int count = analyzed_kernels[short_name]++;
    if (count == 0) nnout() << "running_kernel_" << short_name << "_...";
}
nvbit_enable_instrumented(ctx, p->f, true);

```

`instrument_function` walks the kernel plus every “related function” NVBit knows it might call (`nvbit_get_related_functions` — this includes every template instantiation the compiler linked in as a *possible* callee, which is why a single cuBLAS `nrm2_kernel` launch can enumerate dozens of unrelated-looking function names; see Section 2.7). For each such function it calls `should_instrument`, which is where whitelist/blacklist filtering, sampling, and — critically — `LOG_KERNELS` mode all converge:

```

bool should_instrument(CUcontext ctx, CUfunction f) {
    ...
    enable_instr &= !kernel_logging_enabled; // <-- LOG_KERNELS forces this to false
    if (sampling != 0 && analyzed_kernels.count(func_name)) {
        if (analyzed_kernels[func_name] % sampling != 0) enable_instr = false;
    }
    return enable_instr;
}

```

So `LOG_KERNELS` does not merely add logging on top of the normal instrumentation — it hard-disables `should_instrument` for *every* function, meaning `instrument_function` inserts zero device-side calls anywhere, and the “running kernel [...]” print above never fires either (since `enable_instr` is false). The kernel still runs, at essentially native speed, because `nvbit_enable_instrumented` is called unconditionally and simply makes NVBit run the (uninstrumented, in this case identical-to-original) version.

For every instruction NVBit hands back for a function that *is* selected, `instruction_info::get_reginfo(instr)` decides whether the instruction is floating-point-interesting at all, and if so, what its operands look like.

2.3 The opcode-to-operand table

`get_reginfo` (in `instruction_info.cu`) matches the instruction’s opcode string against a table of regex patterns baked into the binary as `src/instructions.h` — a byte array (`instructions_json`, 5,433 bytes) that is really just a JSON document, embedded at compile time and parsed once at first use with `nlohmann::json`. Two representative entries:

```

"HMMA.16816.F16": {
  "registers": [
    {"type": "f16", "count": 4},
    {"type": "f16", "count": 8},
    {"type": "f16", "count": 4},
    {"type": "f16", "count": 4}
  ],
  "description": "Half-precision matrix multiply accumulate into half-precision accumulator"
},
"DFMA(\\RZ|\\RP|\\RM)?(\\FTZ)?(?:\\.(RP|RM|RZ))?(?:\\S\\.SAT)?": {
  "registers": [
    {"type": "f64", "count": 1},

```

```

    {"type": "f64", "count": 1},
    {"type": "f64", "count": 1}
  ],
  "description": ""
}

```

Each entry says, per operand position, which floating-point format the operand is (**f16/bf16/f32/f64**) and how many scalar values are packed into it (e.g. a tensor-core HMMA operand carrying 8 packed half-precision values). For each such operand, `get_regs` figures out *how* to actually read the value at instrumentation-insertion time — a register, a uniform register, or a constant-bank (**cbank**) immediate — and builds a list of zero-argument closures (**reginsertion**, i.e. `std::function<void()>`) that, when called later, will emit the NVBit call (`nvbit_add_call_arg_reg_val/_ureg_val/_cbank_val`) that actually smuggles that operand’s bits into the injected device function call. A literal double/float immediate operand (**IMM_DOUBLE**) is instead classified right there at instrumentation time via host-side `fpclassify` and, if already NaN/Inf/subnormal, produces an immediate warning line rather than a runtime check (there is nothing to check at run time — the value never changes).

This table is also where the 24-line `is_f64high` check matters: some SASS forms split a 64-bit double across two 32-bit registers and give the *high* half its own opcode suffix (`...64H`); the table’s register-start-offset logic (`reg_start = op->u.reg.num - (f64high ? 1 : 0)`) exists purely to correctly pair the two halves back into one 64-bit value.

2.4 Exception detection, end to end

For every FP-classified instruction (and `INSTRUMENT_EXCEPTIONS` is 1 by default), `instrument_function` in `nixnan.cu` inserts *two* calls to the same device function, `nixnan_check_regs` (defined in `inject_funs.cu`): once `IPOINT_AFTER` (reading the operand *after* it takes its post-instruction guard-predicate value — effectively the output/destination operands) and once `IPOINT_BEFORE` (the input operands). Each call carries, as variadic arguments, a packed `reginfo` struct (a bit-packed 32-bit struct — `count:6`, `type:4`, `half_h0:1`, `half_h1:1`, `div0:1`, `operand:2`, `num_regs:4`, `f64high:1`, statically asserted to fit in one `int32_t`) followed by the actual register values.

`nixnan_check_regs` then, per operand, classifies the raw bits using the IEEE-754 bit-pattern predicates in `fp_utils.cuh` (`half_is_nan`, `float_is_inf`, `double_is_subnorm`, etc. — pure exponent/mantissa bit masking, no floating-point arithmetic at all, so this cannot itself introduce rounding artifacts):

```

uint32_t float_is_nan(uint32_t f) {
    return (float_exp(f) == 0xFF && float_mant(f) != 0) ? E_NAN : 0;
}
uint32_t float_is_inf(uint32_t f) {
    return (float_exp(f) == 0xFF && float_mant(f) == 0) ? sgninf(f) : 0;
}

```

The five exception bits are `E_NAN`, `E_INF`, `E_NINF` (negative infinity, reported separately from positive), `E_SUB` (subnormal), and `E_DIV0` — the last of which is really just “this operand is exactly zero”, only checked for operands the instruction table flags with `"div0": true` (i.e. divisor operands of division-shaped instructions). `half2/bf162` variants unpack two packed half-precision lanes from one 32-bit register before classifying each.

The per-lane exception bitmask is then combined across the whole warp with a `__shfl.sync` reduction (any lane’s exception counts for the warp), and only the warp’s first active lane does the recording, to avoid 32-way redundant writes:

```
if (first_laneid == laneid) {
    for (int op = 0; op < OPERANDS; op++) {
        uint32_t exce = excs[op];
        if (exce != 0) {
            uint32_t num_exceptions = recorder.record(inst_id, exce, op);
            if (num_exceptions == 0) {
                // first time this (instruction, operand) pair has ever
                // hit this exact exception -- push a live report
                for (int skip = 0; skip < 2; skip++) {
                    exception_info ei(get_ctaid(), get_warpid(), inst_id, exce, op, UNKNOWN, skip);
                    channel_dev->push(&ei, sizeof(exception_info));
                }
            }
        }
    }
}
```

`recorder.record (device_recorder::record, recording.h)` is an `atomicAdd` into a flat device array, indexed by (instruction id, operand, exception bitmask). This is the counter that **every** occurrence increments, whether or not a message is pushed anywhere — it is the source of the “repeats” numbers in the final report. The channel push, by contrast, only happens the *first* time a given (instruction, operand, exception-bits) triple is ever seen (`num_exceptions == 0` means this was the very first increment) — this is what produces the live `#nixnan: error [...] detected in operand ...` lines this session saw streamed to the console, and it happens exactly once per distinct site, no matter how many times that site actually fires.

The channel and receiver thread

NVBit tools cannot easily do host-side I/O from device code, so `nixnan.so` uses NVBit’s lock-free `ChannelDev/ChannelHost` ring buffer: device code pushes fixed-size `exception_info` records; a dedicated host `std::thread (recv_thread_fun in nixnan.cu)` polls `channel_host.recv` and, for each record, looks up the recorded instruction’s SASS text, function name, and source location (all previously stashed by `recorder::mk_entry` at instrumentation time) and prints the formatted error line. Every push happens *twice*, with a “skip” flag on the second copy — this is an NVBit idiom to guarantee the channel has fully drained even if the consuming thread is mid-poll (`to_skip()` entries are silently discarded by the receiver).

The final aggregate report

At context teardown (`nvbit_at_ctx_term`), the device-side error array is copied back to the host and folded into per-(format, is_memory) totals. For each of the five exception kinds, the code computes *two* numbers from the same per-instruction/operand counter table:

```
if (exce & error) {
    std::get<0>(exception_counts[{type, is_mem}][index]) += errors; // total occurrences (
        repeats, so far)
    if (errors > 0) std::get<1>(exception_counts[{type, is_mem}][index])++; // one more
        distinct site
```

```

}
...
// later: subtract the per-format unique-site count out of the running total
std::get<0>(exception_counts[{ftype, is_mem}][i]) -= std::get<1>(exception_counts[{ftype
, is_mem}][i]);

```

That subtraction is exactly why the printed report reads NaN: 22 (154 repeats): 22 is the number of distinct (instruction, operand) sites that ever produced a NaN even once, and 154 is *additional* occurrences beyond each site’s first (i.e. total occurrences minus the 22 “first hits” already counted once each in the unique-site tally) — so **unique + repeats** is the true total number of times that exception fired, summed over every site. This is the exact metric this session used to compare `rd_nixnan.cu` against its 10x-grid-point variant: unique-site count is a static property of the kernel’s source code and stays fixed as the problem scales, while repeats measures how much bad floating-point behavior an actual run produced, and that number does scale with problem size.

2.5 Memory-instruction checking

If `INSTR_MEM=1`, every global-memory store (STG, any width) additionally gets a call to `nixnan_check_nans` (`memcheck.cu`) inserted `IPOINT_BEFORE`. `meminstrumentation.cu` first has to work out *what type* is being stored, since a raw STG instruction’s operand is just an untyped register: it walks backwards through the current basic block looking for the most recent instruction that wrote that register and reuses *that* instruction’s already-known operand type (`find_type`). If no typed producer is found, the store is skipped rather than guessed at. Unlike the exception path above, the memory-instruction checker only ever reports NaN (not Inf/Subnormal/Div0) — this is why every “...Memory Operations” block in the report only ever prints a NaN: line.

2.6 The exponent-histogram / bin-threshold mechanism

This is the machinery behind `HISTOGRAM/BIN_SPEC_FILE`, and it is what `autonixnan`’s “extreme-value scan” phase (Section 3.3) is actually built on. It runs as a *parallel, independent* instrumentation pass alongside the exception checker above — every FP instruction that gets a `nixnan_check_regs` call also, unconditionally, gets a call to `nixnan_fp_histogram_counter` inserted (`fp-histogram.cu`); whether that call does anything is gated at run time by `histogram_enabled` (true if `HISTOGRAM=1` or `BIN_SPEC_FILE` is set).

`BIN_SPEC_FILE` is a small JSON document naming, per format, a list of [`lower`, `upper`] *unbiased signed exponent* ranges to watch, plus a `count` (report every `count`-th value landing in a watched bin) and optional `max_reports` (self-terminate a kernel-function’s reporting after this many). If the named file does not exist, `nixnan.so` writes an empty template there and `exit(0)`s — by design, the first invocation with a not-yet-existing spec file is a no-op setup step, not a real run.

At run time, `record()` (in `fp-fun.cu`) does two things for every FP value seen: it always `atomicAdds` into a flat [`format`] [`exponent`] histogram array (this is what powers the “Exponent range for f16: [-14, 15]” summary line printed at `term()`, when `HISTOGRAM=1` is used directly rather than through `autonixnan`), and, if that exponent falls in one of the function’s configured bins, increments a per-bin counter and — every `count`-th time — pushes a **bin has reached threshold** report through its own separate channel/receiver-thread pair (structurally identical to the exception path, but entirely separate state). **Crucially, the valid exponent range for each format explicitly excludes zero and infinity** (see `FP_FORMAT_RANGES` in `bin/autonixnan`, and `bin_from_json`’s

range validation in `fp-histogram.cu`, which rejects any bin touching the all-zeros/all-ones exponent encodings) — **a value that jumps straight from a normal, finite magnitude to Inf in a single instruction never lands in any bin at all, and is therefore invisible to this mechanism.** This is exactly what this session found empirically with the mixed-precision-Cholesky Phase-D overflow experiment (Section 4): the scan flagged an unrelated fp64 QR kernel while missing the real, program-confirmed FP16 overflow entirely, because that overflow was a one-step jump to Inf with no gradually-climbing finite exponent trail leading up to it.

2.7 LOG_KERNELS mode

As established above, LOG_KERNELS disables all instrumentation. What it *does* do: at kernel-exit time, `nvbit_at_cuda_event` prints `Kernel [name] execution time: N microseconds` for every launch (gated on `time_kernels || kernel_logging_enabled`, so this line’s presence does not actually require `TIME_KERNELS=1` once LOG_KERNELS is set — `autonixnan`’s `collect_kernels()` relies on exactly this). Separately, `instrument_function` also prints one `Function [related] of kernel [top-level] line for every NVBit-enumerated related function of a kernel`, which for a library-heavy binary (cuBLAS/cuSOLVER template instantiations) can be enormous — an `n=4096` cuSOLVER Cholesky factorization run in this session produced over 10,000 such lines, none of which match `autonixnan`’s `#nixnan: Kernel [...] execution time:` parser regex and are therefore silently ignored by `_parse_kernel_log`. They are harmless noise from `autonixnan`’s point of view, but they do inflate the log file and are worth knowing about if reading a LOG_KERNELS log by hand.

2.8 The (currently) dead NIXNAN_TIMEOUT

`src/timer.h/timer.cu` implement a self-contained, graceful internal timeout: `nixnan::timer::init()` reads `NIXNAN_TIMEOUT` (default 0, meaning disabled), and `nixnan::timer::tool_init(ctx)`, if the timeout is positive, spawns a background thread that sleeps for that many seconds and then calls `nnterminate("Timeout reached.")` — a graceful, in-process SIGINT to itself, flushing the log first.

This code is never called. A full read of `nixnan.cu`’s `nvbit_at_init`, `nvbit_tool_init`, and `nvbit_at_ctx_term` — the only three places that call into any other namespace’s `init/tool_init/term` — shows calls into `nixnan::fp_histogram::{init,tool_init,term}` only. There is no call anywhere in the tree into `nixnan::timer::{init,tool_init,term}`. This was confirmed two independent ways during this session:

- `grep -rn "timer::" src/` outside of `timer.cu` itself returns nothing.
- The `GET_VAR_INT` macro used to read `NIXNAN_TIMEOUT` *always* prints a banner line when it runs, regardless of the value found. Every banner captured this session (which comes from `nvbit_at_init`, run unconditionally) lists `INSTR.BEGIN` through `FUNCTION.BLACKLIST` — and never `NIXNAN_TIMEOUT`. If the variable were actually being read, its banner line would be there.

The practical consequence: `bin/autonixnan`’s `find_extreme_values()` sets `NIXNAN_TIMEOUT` and documents (in its own docstring) that this is what lets “the target’s own timer, rather than the hard kill here” end a long-running scan gracefully. In the currently built `nixnan.so`, that never happens — `NIXNAN_TIMEOUT` is silently ignored, and *every* timeout this session observed (“Error: target program exceeded Ns timeout”) was Python’s own `subprocess.run(timeout=...)` hard-killing the child with SIGKILL after the full `args.timeout + TIMEOUT_GRACE` (30s) window, not a graceful internal exit. This does not make `autonixnan` incorrect — the external kill still bounds run time

correctly — but it does mean the 30-second grace period is not, in practice, “slack given in case the internal timer is a little late”; it is dead weight, since nothing internal is ever going to fire first.

3 Part II — bin/autonixnan

`autonixnan` runs three phases against the same target command, in sequence, unconditionally launching the target process fresh each time (there is no persistent daemon or warm cache between phases).

3.1 Phase 1: `collect_kernels()` — the baseline

```
env["LD_PRELOAD"] = lib_path
env["LOG_KERNELS"] = log_file
elapsed = run_with_env(program_args, timeout, env)
```

This runs the target with instrumentation fully disabled (Section 2.7), so wall time here is close to native (modulo the overhead of NVBit’s own related-function enumeration and the “Function [...] of kernel [...]” print spam for library-heavy binaries, which this session measured at 10-17s for a cuSOLVER-based binary that runs in under 2s uninstrumented). The `-t/--timeout` value is used *directly, with no grace period*, as the hard `subprocess.run` timeout for this phase — a target that is merely slow to start (library init, matrix setup) can therefore fail here even though nothing is actually being instrumented, which this session hit directly with a cuSOLVER-heavy binary whose factorization setup alone exceeded a naively small `-t`.

The resulting log is parsed by `_parse_kernel_log`, which only matches lines starting “#nixnan: Kernel [” — ignoring the “Function [...] of kernel [...]” noise described above — extracting the kernel name and, if present, its microsecond execution time, and building both an in-order call sequence and a name → {count, total_time} table.

3.2 Report assembly (between phases 1 and 2)

Before running Phase 2, `main()` prints the “Unique Kernels” table (sorted by total time descending, truncated to the top 25 when there are more), the baseline wall-clock time, and the full kernel call sequence — elided to the first and last 25 calls with a “(...calls omitted)” marker in between when there are more than 50 total, so an `n=4096` factorization’s 900+ launches do not flood the terminal.

3.3 Phase 2: `find_extreme_values()` — the exponent scan

```
bs = _make_binspec(1, max_reports) # count=1: report every single hit, capped at
    max_reports/kernel
env["BIN_SPEC_FILE"] = spec_file
env["LOGFILE"] = log_file
env["NIXNAN_TIMEOUT"] = str(timeout) # see Sec 2.9 -- currently a no-op
env["INSTRUMENT_EXCEPTIONS"] = "0" # exception checker off; histogram checker still runs
elapsed = run_with_env(program_args, timeout + TIMEOUT_GRACE, env, check=False)
```

`_make_binspec` auto-generates a `BIN_SPEC_FILE` that watches the top 5% and bottom 5% of *every* format’s valid (finite, non-zero) exponent range, for *every* function — since the spec file has no per-function key, `fp-histogram.cu`’s `process_bin_spec` applies the same global spec to whichever

function it is asked about, so in effect every instrumented kernel is watched. This is a real, full NVBit exception-instrumentation-grade pass over the entire binary (with the exception checker itself switched off via `INSTRUMENT_EXCEPTIONS=0`, leaving only the histogram counter active) and is therefore subject to the same heavy per-distinct-kernel overhead as any other full instrumentation run — this session measured it taking well over a minute for a cuSOLVER/cutlass-heavy binary with hundreds of distinct kernel templates, versus milliseconds for a handful of custom kernels called thousands of times.

`run_with_env` is called with `check=False`, so a non-zero exit (which is the *expected*, normal outcome here — the target hits `max_reports` for some kernel and `nnterminates` itself with `SIGINT`) is not treated as an error; only an actual `subprocess.TimeoutExpired` (after `timeout + 30s`) is fatal to this phase.

The resulting log is scanned by `BIN_REACHED_RE` for `"#nixnan: <fmt> bin has reached threshold: function=...range=[lo,hi] count=N"` lines, tallied per function name, and the single function with the most such reports across the whole run is chosen as “the” function of interest.

3.4 Phase 3: `run_instrumented()` — the deep dive

If Phase 2 found anything, `autonixnan` runs the target one final time, with `LD_PRELOAD` set and *no other environment variable* beyond a `FUNCTION_WHITELIST` file containing exactly the one function name Phase 2 flagged:

```
env["LD_PRELOAD"] = lib_path
if whitelist_file:
    env["FUNCTION_WHITELIST"] = whitelist_file
elapsed = run_with_env(program_args, timeout, env, quiet=False)
```

Because no other variables are set, this run gets `INSTRUMENT_EXCEPTIONS`’s *default* of 1 (full exception checking, Section 2.4) and `HISTOGRAM`’s default of 0 (no histogram) — but scoped, via the whitelist, to *only* the one function Phase 2 flagged (plus, per `should_instrument`, any function whose *exact* name appears in that whitelist; unrelated kernels are left completely uninstrumented, running at native speed). `quiet=False` means this is the one phase whose child process’s `stdout/stderr` are not suppressed — this is why the live `#nixnan: error [...]` lines and the final `----- nixnan Report -----` aggregate (Section 2.4) appear directly in `autonixnan`’s own output during this phase and nowhere else.

This phase’s own log file is opened and unconditionally `os.unlinked` in a `finally` block *without ever being read* — `LOGFILE` is never set for this call (it is commented out in the source), so the target’s `nnout()` output goes to `stderr`, which `quiet=False` already passes through to the console directly; the temp file exists only so `tempfile.mkstemp` has something to clean up, and `run_instrumented`’s returned `kernels_seq/kernels_unique` are consequently always empty and are not used by `main()`.

This is also the phase where the exponent-scan/exception-checker scope mismatch documented in Section 2.6 bites hardest: if the real fault is a hard jump straight to `Inf` (rather than a gradual climb through extreme-but-finite exponents), Phase 2 will not flag the kernel where it actually happens, so Phase 3’s whitelist will scope full exception-instrumentation to the *wrong* kernel, and the final aggregate report can show all-zero counts in every format even though the target program’s own internal exception detection (if it has any) clearly fired. Section 4 gives the exact reproduction from this session.

3.5 Timeout semantics, summarized

Phase	Hard kill after	Notes
1: baseline	<code>-t</code> exactly	No grace period; catches slow startup too.
2: exponent scan	<code>-t + 30s</code>	Grace period, currently pointless (Sec. 2.8).
3: deep dive	<code>-t</code> exactly	Same value re-used; not separately configurable.

Every phase re-uses the *same* `-t` value from the command line; there is no way to give the baseline phase a short budget and the scan phase a long one from the CLI as it stands.

4 Empirically Observed Limitations

These are not code-reading conclusions but things directly reproduced against real binaries this session, cross-checked against the source above.

1. **Hard overflow jumps are invisible to the extreme-value scan.** Running a mixed-precision Cholesky iterative-refinement solver (`SC26-expts/mpir_gpu`) with its documented overflow-forcing flags (`--noscale --bscale 1e5`) produced a program-confirmed FP16 overflow (`nonfinite=256` from the target’s own internal check) on the very first iteration. `autonixnan`’s Phase 2 never flagged the kernel responsible; it flagged an unrelated fp64 QR kernel used only during test-matrix generation, and Phase 3’s whitelisted full-instrumentation pass consequently reported zero FP16 exceptions. Running `nixnan.so` directly (no `autonixnan`, no whitelist, default `INSTRUMENT_EXCEPTIONS=1`) against the same invocation found the real exceptions immediately — filed under the FP32 bucket, not FP16, because that solver configuration only rounds values into fp16’s representable range via a cast; the arithmetic that actually overflows executes as native FP32 SASS instructions. Neither of these is a `nixnan` bug: the histogram mechanism’s zero/inf exclusion is by design (Section 2.6), and the FP32 attribution is simply an accurate report of which SASS instruction type executed on the bad value.
2. **The dead internal timeout** (Section 2.8) means every `-t` this session used was enforced purely externally, with a pointless 30s grace period tacked onto the scan phase.
3. **LOG_KERNELS noise scales with library surface area, not with actual kernel diversity.** A `cuSOLVER/cuBLAS`-heavy binary produced 10,000+ log lines from related-function enumeration alone, none of which affect `autonixnan`’s parsed output but which do make the raw log file unwieldy to read by hand.

5 Practical Summary

To reproduce any given phase by hand outside `autonixnan`, the minimal commands are:

```
# Phase 1 equivalent -- kernel catalog, no instrumentation
LD_PRELOAD=./nixnan.so LOG_KERNELS=/tmp/k.log ./target [args...]

# Phase 2 equivalent -- exponent-extreme scan, exceptions off
LD_PRELOAD=./nixnan.so BIN_SPEC_FILE=/tmp/spec.json LOGFILE=/tmp/scan.log \
  INSTRUMENT_EXCEPTIONS=0 ./target [args...]

# Phase 3 equivalent, or a direct full exception run (no whitelist
```

```
# needed if the binary is small/fast enough to instrument in full)
LD_PRELOAD=./nixnan.so ./target [args...]
```

The last form – direct, unwhitelisted `nixnan.so`, no `autonixnan` involved – is the one to reach for whenever a target is small enough to instrument in full and the exponent-scan’s blind spot (hard overflow jumps) is a concern, since it is the only one of the three that does not depend on Phase 2 having correctly identified the kernel of interest.